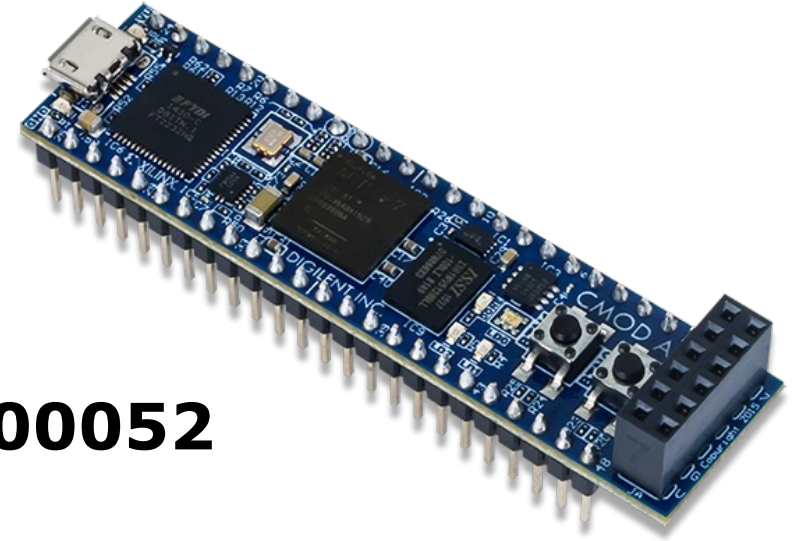


Lorenzo Pina 894396, Simone Rancati 900052

Informatica Industriale



l.pina4@campus.unimib.it, s.rancati8@campus.unimib.it
<https://github.com/LorenzoPinaUnimib/InformaticaIndustriale>

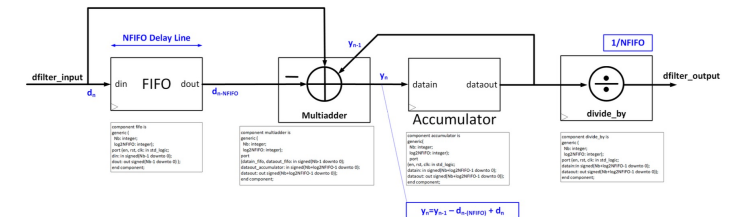
Microelectronics Group



Introduction

```
generic(  
    Nbf      : integer;  
    log2NFIFO : integer  
);  
port(  
    clk      : in  std_logic;  
    en       : in  std_logic;  
    rst      : in  std_logic;  
    datain   : in  unsigned(Nbf-1 downto 0);  
    dataout  : out unsigned(Nbf+log2NFIFO-1 downto 0)  
);
```

- ❑ Filter_MA uses datain and dataout as input and output signals
 - ❑ Other signals are used internally to communicate between components
 - ❑ The parameters used are Nbf and log2NFIFO
- Nbf = bit resolution of the original signal**
log2NFIFO = log2 of the number of elements in the FIFO



```
u_fifo: fifo  
generic map(Nb => Nbf, log2NFIFO => log2NFIFOf)  
port map(clk => clk, en => en, rst => rst,  
    din => datain, dout => dout_fifo);  
  
u_multiadder: multiadder  
generic map(Nb => Nbf, log2NFIFO => log2NFIFOf)  
port map(datain_fifo => datain,  
    dataout_fifo => dout_fifo,  
    dataout_accumulator => dout_accumulator,  
    dataout => dout_multiadder);  
  
u_accumulator: accumulator  
generic map(Nb => Nbf, log2NFIFO => log2NFIFOf)  
port map(clk => clk, en => en, rst => rst,  
    datain => dout_multiadder,  
    dataout => dout_accumulator);  
  
u_divide_by: divide_by  
generic map(Nb => Nbf, log2NFIFO => log2NFIFOf)  
port map(clk => clk, en => en, rst => rst,  
    datain => dout_accumulator,  
    dataout => dataout);
```

Design of Multiadder

```
architecture behavioral of multiadder is

    signal datain_fifo_large  : unsigned(Nb+log2NFIFO-1 downto 0);
    signal dataout_fifo_large : unsigned(Nb+log2NFIFO-1 downto 0);
    signal mask0              : unsigned(log2NFIFO-1 downto 0) := (others => '0');

begin
    datain_fifo_large <= mask0 & datain_fifo;
    dataout_fifo_large <= mask0 & dataout_fifo;

    -- Equazione ricorsiva del filtro a media mobile
    dataout <= dataout_accumulator + datain_fifo_large - dataout_fifo_large;
end behavioral;
```

- ❑ We use mask0 to extend datain_fifo and dataout_fifo to Nb+log2NFIFO-1 bits
- ❑ We calculate the sum of the last NFIFO elements by taking the last result (saved in the accumulator), removing the last element and adding the new one

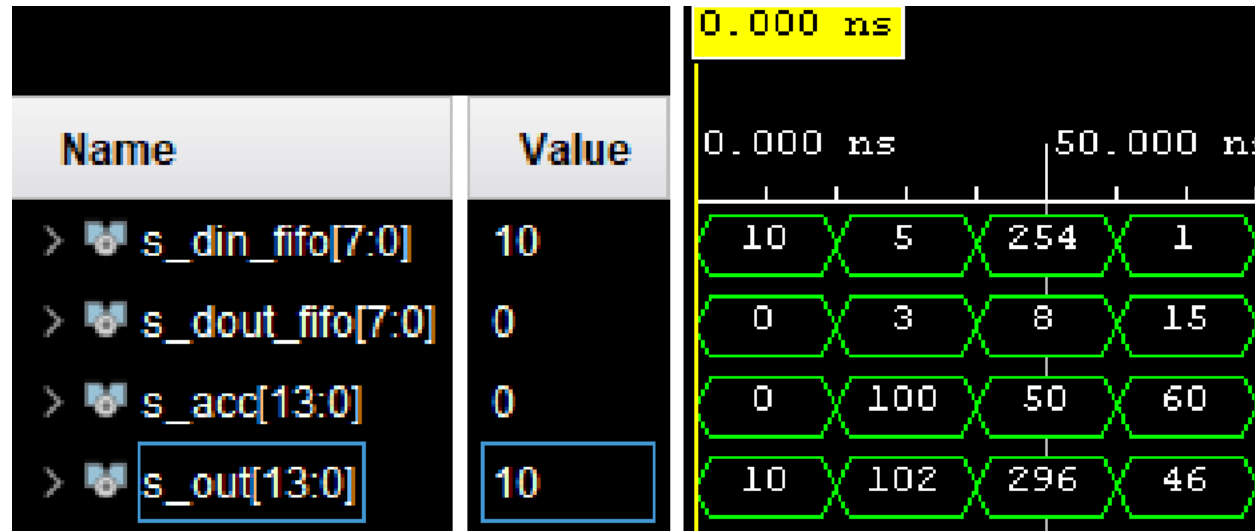
Design of Multiadder - Testbench

```
begin
  -- Test 1: 0 + 10 - 0 = 10
  s_acc      <= to_unsigned(0,  Nb+log2NFIFO);
  s_din_fifo <= to_unsigned(10, Nb);
  s_dout_fifo <= to_unsigned(0,  Nb);
  wait for 20 ns;

  -- Test 2: 100 + 5 - 3 = 102
  s_acc      <= to_unsigned(100, Nb+log2NFIFO);
  s_din_fifo <= to_unsigned(5,  Nb);
  s_dout_fifo <= to_unsigned(3,  Nb);
  wait for 20 ns;

  -- Test 3: 50 + 254 - 8 = 296
  s_acc      <= to_unsigned(50,  Nb+log2NFIFO);
  s_din_fifo <= to_unsigned(254, Nb);
  s_dout_fifo <= to_unsigned(8,  Nb);
  wait for 20 ns;

  -- Test 4: 60 + 1 - 15 = 46
  s_acc      <= to_unsigned(60, Nb+log2NFIFO);
  s_din_fifo <= to_unsigned(1,  Nb);
  s_dout_fifo <= to_unsigned(15, Nb);
  wait for 20 ns;
end process;
```



❑ We test the Multiadder by doing some math operations

Design of Accumulator

```
begin
  reg: reg_en_rst
    generic map(Nbit => Nacc)
    port map(
      clk      => clk,
      en       => en,
      rst      => rst,
      datain   => std_logic_vector(datain),
      dataout  => reg_out
    );

  dataout <= unsigned(reg_out);
end structural;
```

```
begin
  -- Mux combinatorio 3:1 per ogni bit: nessun process
  gen_mux: for i in 0 to Nbit-1 generate
    D_next(i) <= '0'          when rst = '0' else
                datain(i)     when en  = '1' else
                Q_int(i);

  end generate;

  -- Nbit flip-flop D in parallelo
  gen_ff: for i in 0 to Nbit-1 generate
    ff_i: dff port map(
      D   => D_next(i),
      clk => clk,
      Q   => Q_int(i),
      Qn  => open
    );
  end generate;

  dataout <= Q_int;
end structural;
```

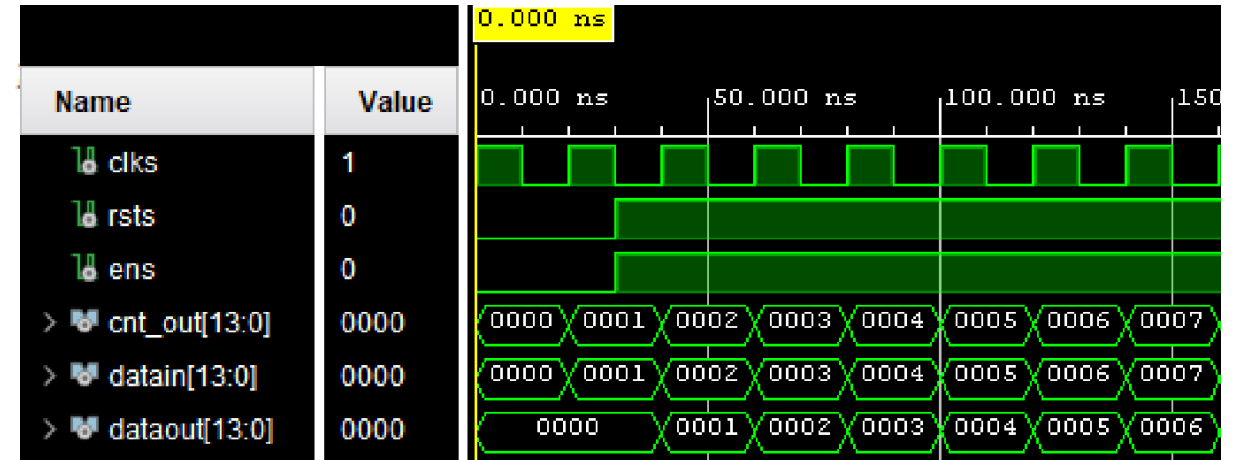
- ❑ We used a D-Flip-Flop to delay the result of the Multiadder by a clock tick
- ❑ The Accumulator resets when the reset signal is 0

Design of Accumulator – Testbench

```
-- Counter: genera 0,1,2,3,... -> datain cresce a ogni clock
dut_counter: counter
    generic map(Nb => Nacc)
    port map(T => '1', clk => clk, OUT_COUNT => cnt_out);

datain <= unsigned(cnt_out);

-- DUT: dataout deve essere uguale a datain del ciclo
dut_acc: accumulator
    generic map(Nb => Nb, log2NFIFO => log2NFIFO)
    port map(clk => clk, en => ens, rst => rst,
        datain => datain, dataout => dataout);
```



- ❑ We test the Accumulator using a counter and seeing if the output is delayed by a clock cycle

Design of Divide_by

```
begin
    -- Shift puramente combinatorio: nessun processo, solo cablaggio
    datain_shifted <= shift_right(datain, log2NFIFO);

    reg: reg_en_rst
        generic map(Nbit => Nacc)
        port map(
            clk      => clk,
            en       => en,
            rst      => rst,
            datain   => std_logic_vector(datain_shifted),
            dataout  => reg_out
        );

    dataout <= unsigned(reg_out);
end structural;
```

- ❑ We implement Divide_by using a right shift of log2NFIFO bits and delaying the output using a D-Flip-Flop

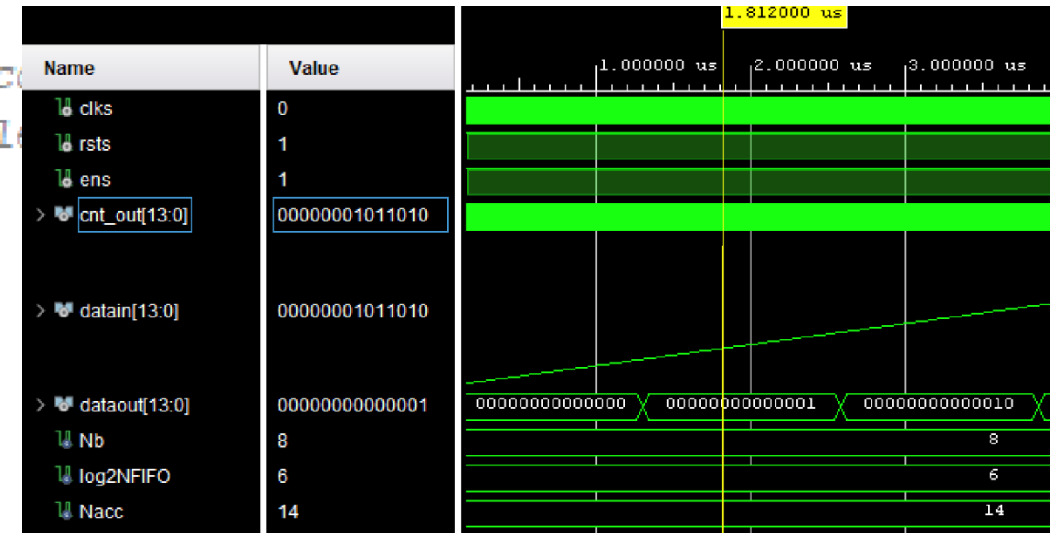
Design of Divide_by - Testbench

```
dut_counter: counter
  generic map(Nb => Nacc)
  port map(T => '1', clk => clks, OUT_COUNT => cnt_out);

datain <= unsigned(cnt_out);
```

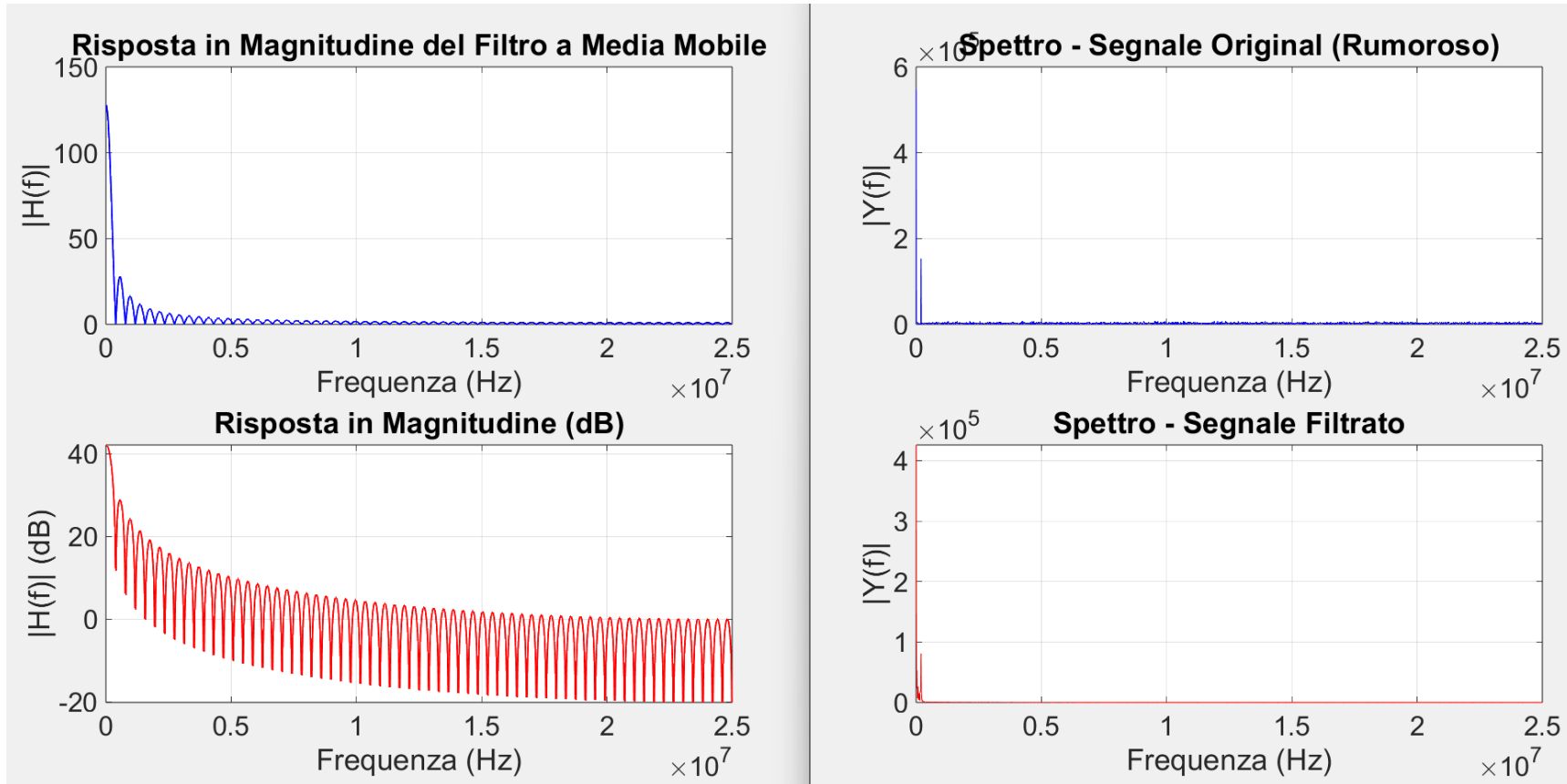
```
-- DUT: dataout = datain >> log2NFIFO (shift aritmetico)
-- Quando datain raggiunge 64 (0x40), dataout deve valere 0
```

```
dut_div: divide_by
  generic map(Nb => Nb, log2NFIFO => log2NFIFO)
  port map(clk => clks, en => ens, rst => rst,
    datain => datain, dataout => dataout);
```



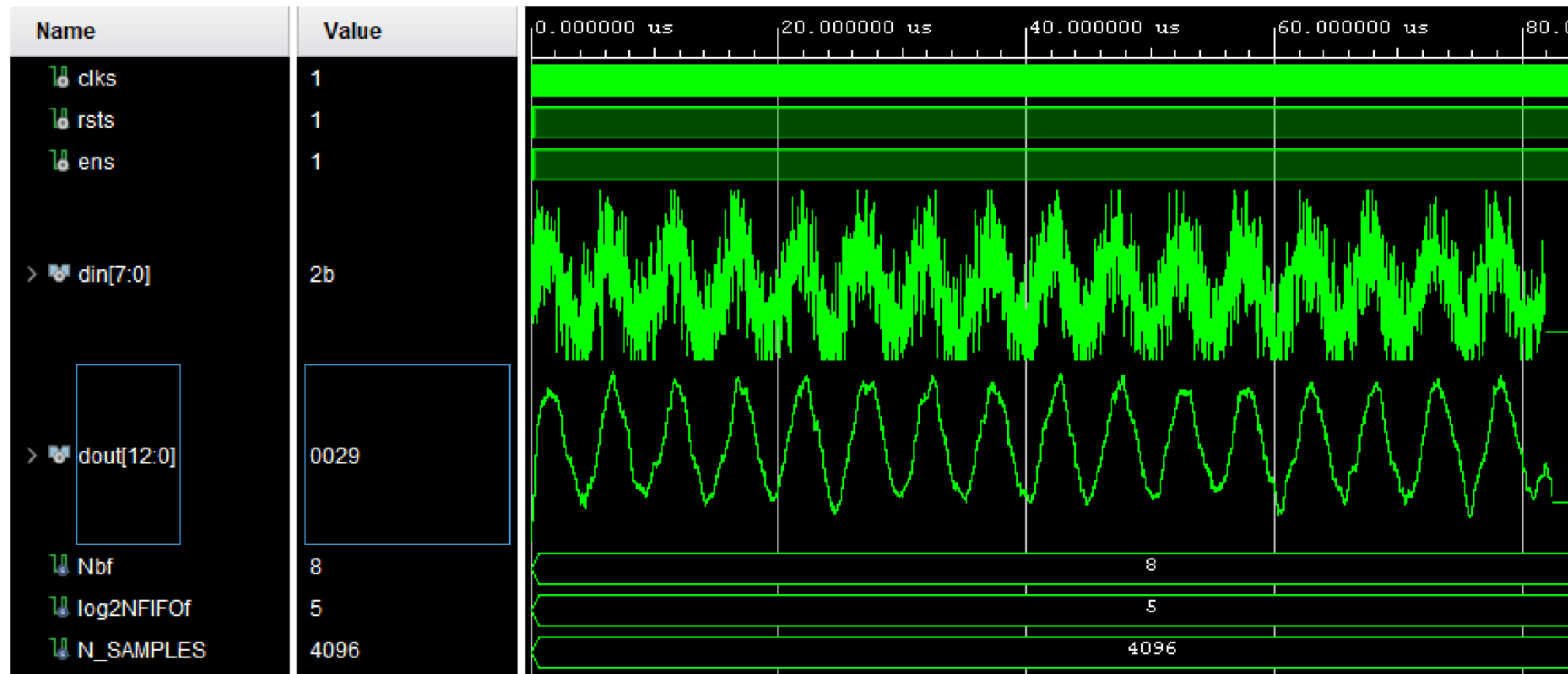
❑ We use a counter to apply the Divide_by to every number

Frequency Response



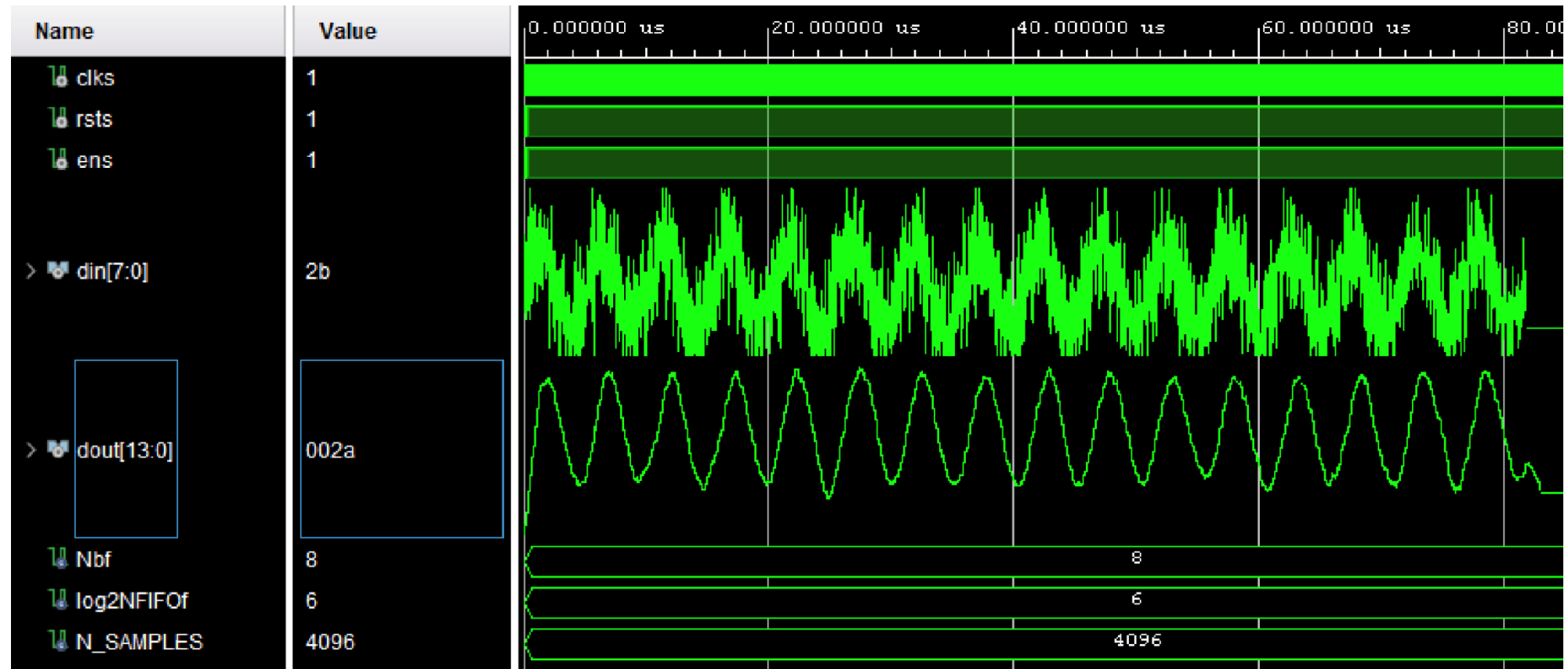
□ The filter has removed the high frequency noise

Simulation result – NFIFO = 32 ($\log_2 \text{NFIFO} = 5$)



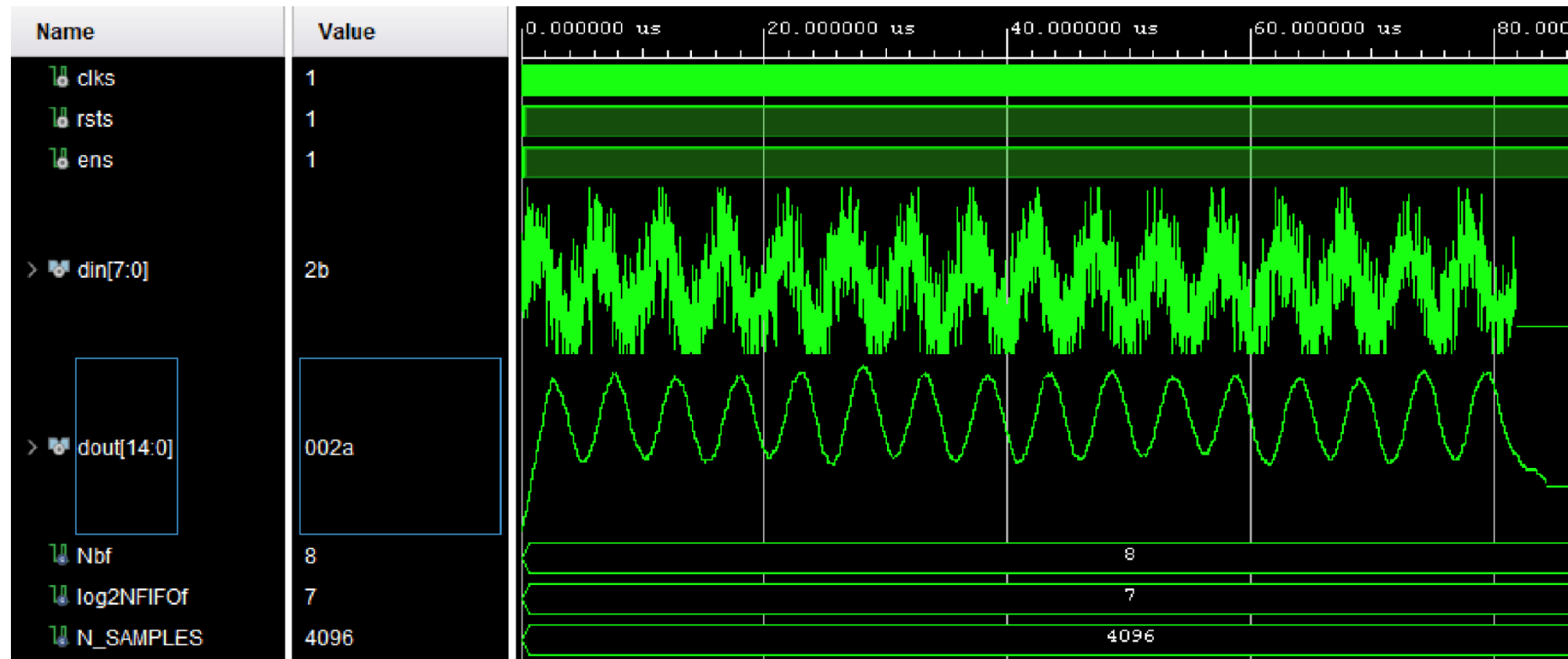
❑ We can see that the result still shows the presence of some noise

Simulation result – NFIFO = 64 ($\log_2 \text{NFIFO} = 6$)



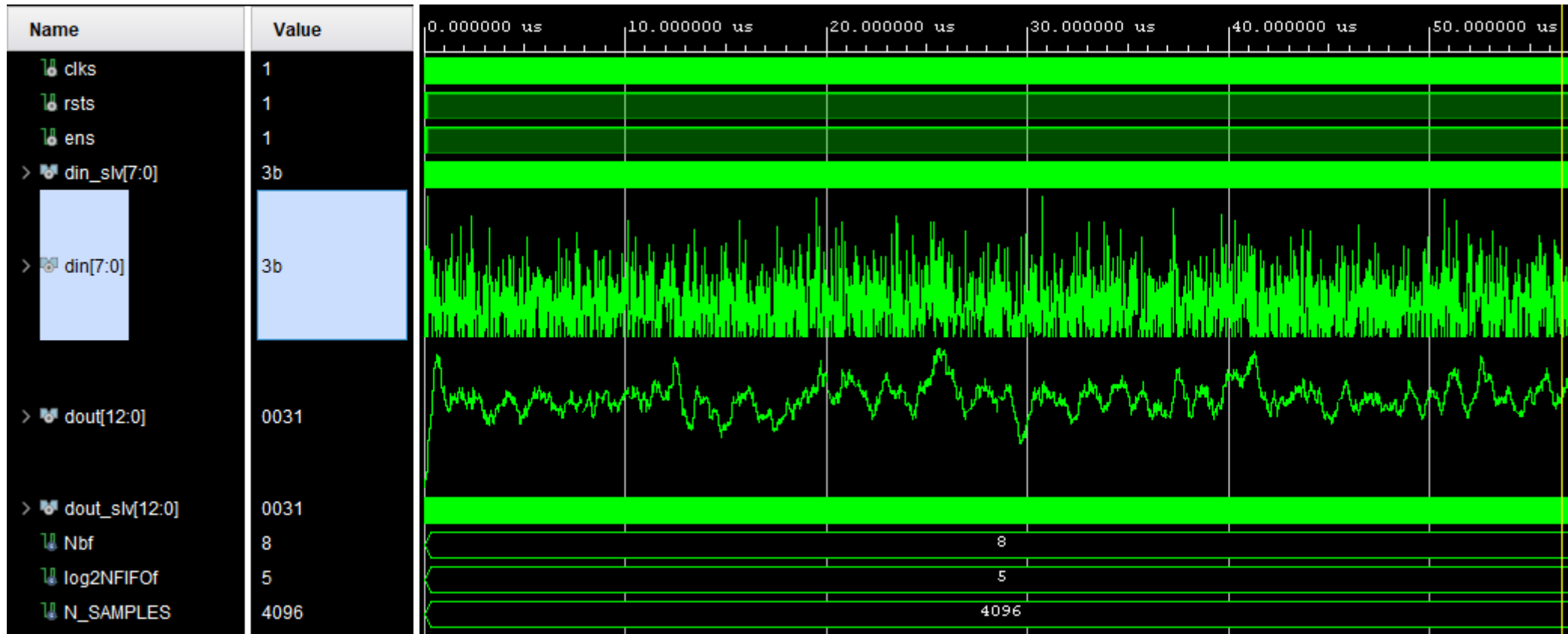
□ With NFIFO = 64 the resulting sinusoid shows an improvement

Simulation result – $\text{NFIFO} = 128$ ($\log_2 \text{NFIFO} = 7$)



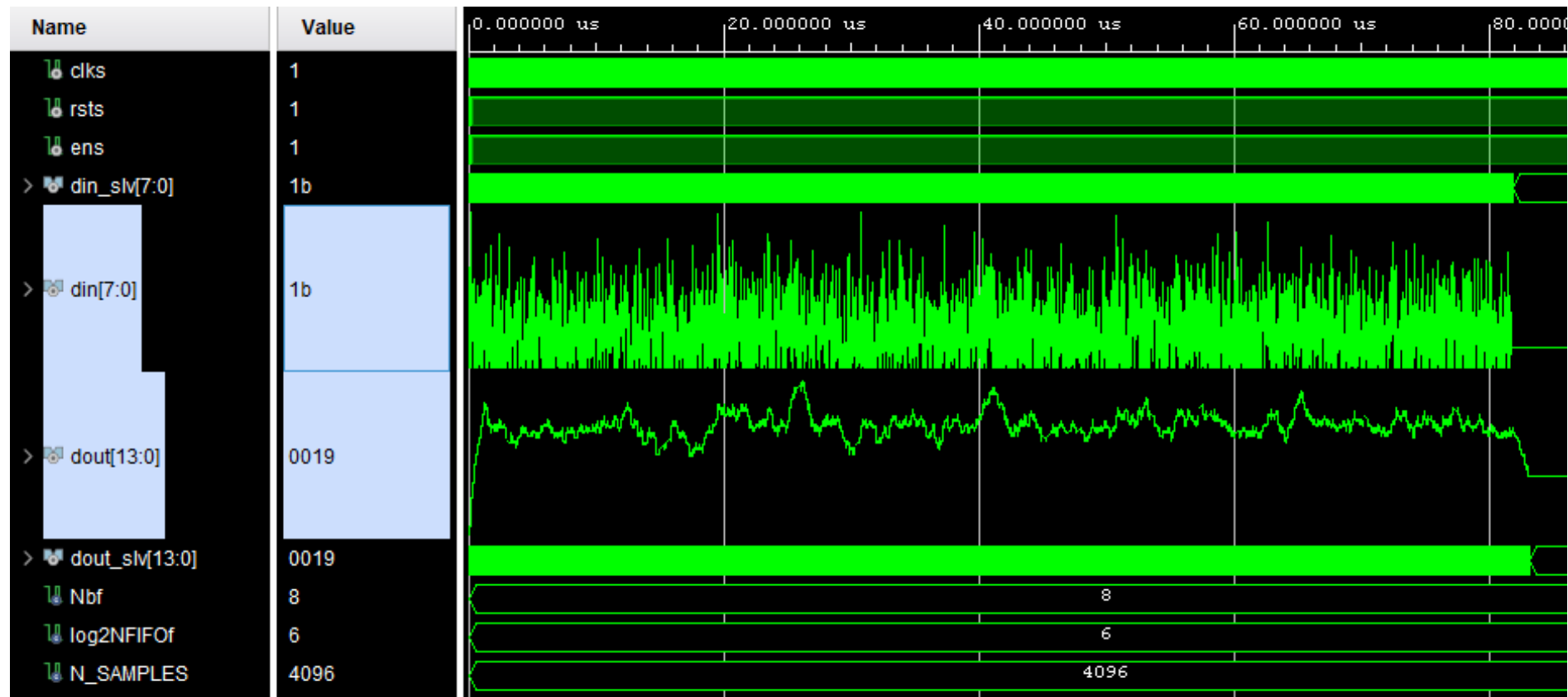
❑ With $\text{NFIFO} = 128$ the sinusoid is smoother but with more attenuation

Simulation result of noise - $NFIFO = 32$ ($\log_2 NFIFO = 5$)



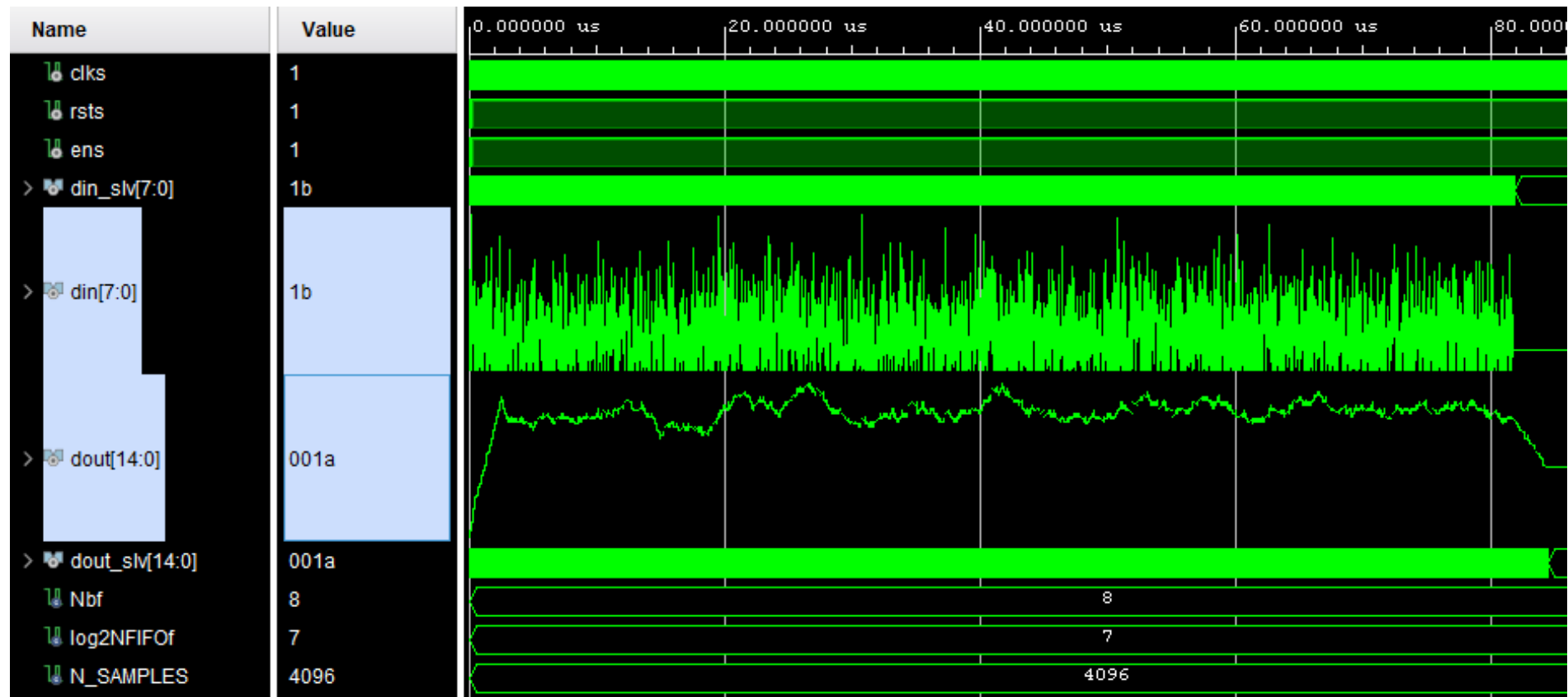
❑ The filter tries to clean the noise signal but the result is not promising

Simulation result of noise - $NFIFO = 64$ ($\log_2 NFIFO = 6$)



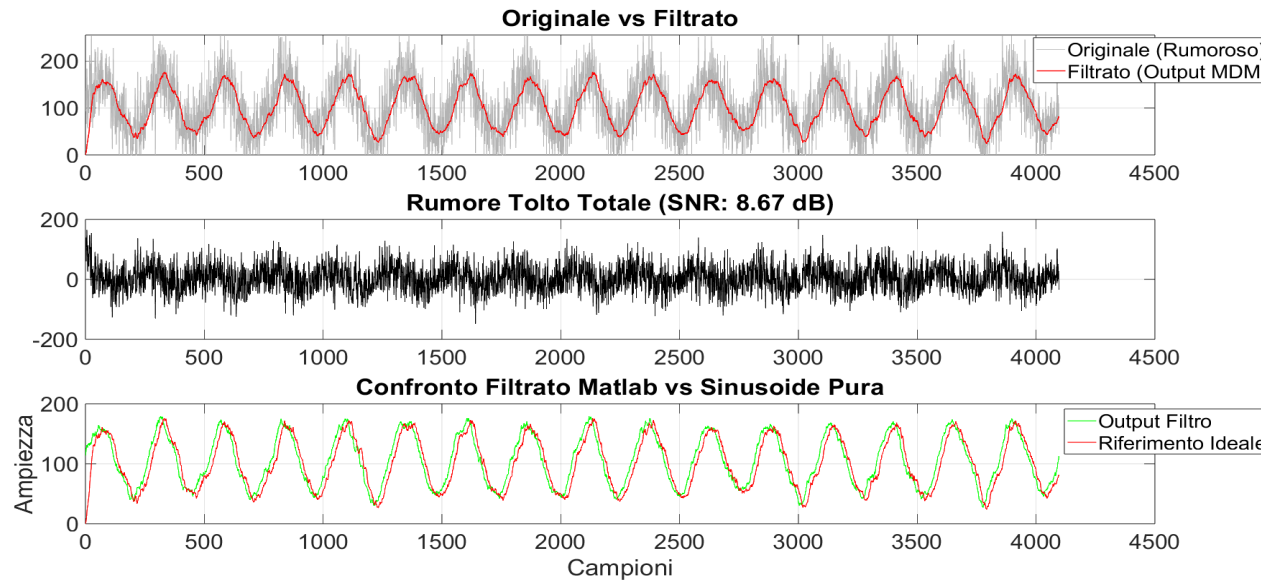
□ With $NFIFO = 64$ we see a noticeable improvement in the resulting signal

Simulation result of noise - $NFIFO = 128$ ($\log_2 NFIFO = 7$)

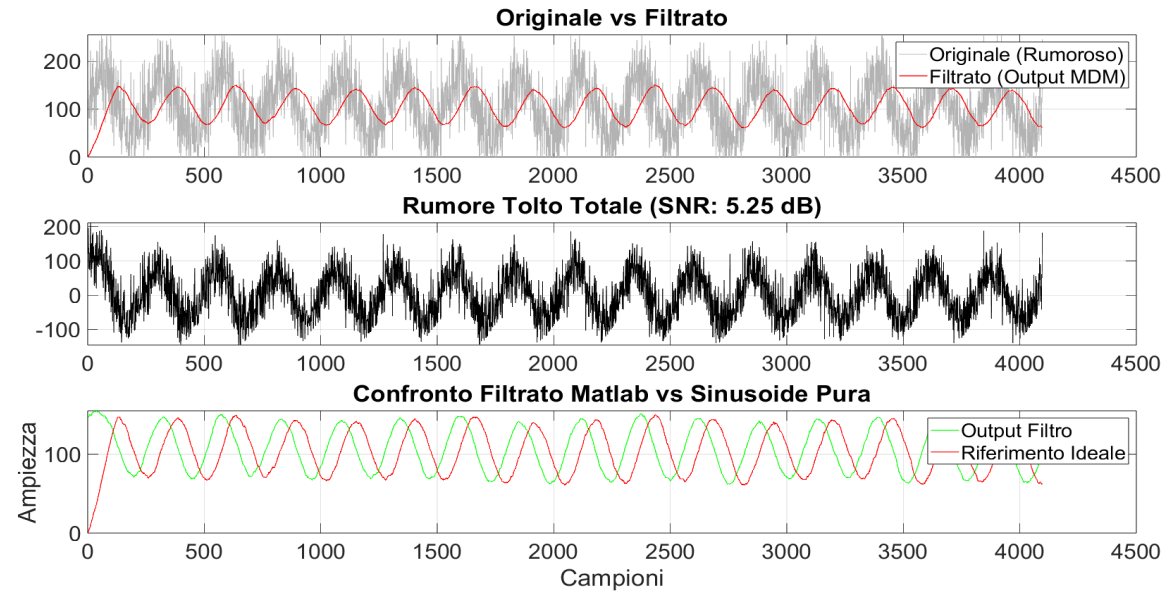


❑ With $NFIFO = 128$ noise is more attenuated but the starting delay is worse

Signal to noise ratio



NFIFO = 32



NFIFO = 128

- ❑ With NFIFO = 32 we notice more noise, less delay and more amplitude
- ❑ With NFIFO = 128 the noise is reduced with a penalization of more delay and less amplitude

Final Table Resume and Comments

NFIFO	SNR	Delay (clock cycles)	Amplitude
32	8,63	34	110
64	7,30	66	100
128	5,25	130	80

- ❑ With the variation of NFIFO we get an improvement in SNR, but there are also unwanted regressions in the amplitude and delay
- ❑ All the tests were done with the Matlab variable SNRdb = 0

Conclusions

- ❑ We concluded that having a filter that cleans more noise can also attenuate the original signal, so it's not always better
- ❑ We tried implementing an EMA (Exponential Moving Average) filter to reduce the delay

```
architecture Behavioral of EMA_filter is
    signal acc : unsigned(WIDTH-1 downto 0) := (others => '0');
begin
    process(clk)
        variable diff : unsigned(WIDTH-1 downto 0);
    begin
        if rising_edge(clk) then
            if rst = '0' then
                acc <= (others => '0');
            else
                if data_in > acc then
                    -- Se l'input è maggiore, la differenza è positiva
                    diff := data_in - acc;
                    acc <= acc + resize(shift_right(diff, K), WIDTH);
                elsif data_in < acc then
                    -- Se l'input è minore, la differenza è negativa
                    -- Sottraiamo la parte frazionaria per scendere verso il valore
                    diff := acc - data_in;
                    acc <= acc - resize(shift_right(diff, K), WIDTH);
                end if;
                -- Se data_in = acc, non facciamo nulla (acc rimane invariato)
            end if;
        end if;
    end process;
end process;
```

